pow(x, y) → x^y

sqrt(x) → $x^{1/2}$

floor(x) → rounds down to the nearest whole number.

floor(12.999) → 12

abs(x) → returns positive value

ceil(x) → rounds up to the nearest whole number

ceil(12.3) → 13

floor(-3.7) → -4

ceil(-3.7) → -3

User defined functions

- Value-returning function
- Void (donot use a return statement)

note:

function definition means fully making the function by also writing code in its body and return type

```
int abs (int num)
{
    if (num < 0)
        num = -num;
    return num;
}
```

function defination

heading.

formal para. (Input)

formal para of abs is num.

```
int abs (int num)
{
    if (num < 0)
        num = -num;
    return num;
}
```

heading.

Function is only executed when we call it.

Syntax

datatype identifier ..... ↑ contains

```
return type function-name ( formal parameters )
{
    // block of code → only executes when we call this.
}
```

```
double pow ( double base, double exponent);
double u = 2.5;
double v = 3.0;
double x, y;
x = pow (u, v);
y = pow ( 2.0, 3.2) + 5.1;
```

calls the function. value of u is copied into base and value of v is copied into exponent.

u, v, 2.0, 3.2 → Actual parameters.

Function call :-

functionname (actual parameter list...);

↓

if any

if formal parameter list is empty then the Syntax is:-

functiontype functionname ( );

if formal parameter list is empty, then the actual parameter list is also empty in a function call.

To call a function with empty formal.p

Syntax

functionname( );

note:- In a function call →

no.of actual parameters = no of formal parameters

note:- A value-returning function is called in a => or output or parameter to another f.

return statement is used to pass the value after processing outside the function.

Syntax

return expr; can be constant
↳ expression
variable

E.g A function that determines larger of two numbers.

```
int Large (int a, int b)        → formal parameters
{
    if (a > b)
      return a
    else
      return b
}

int x, y;                       → actual parameters
cout<< "value of x";
cin>>x;
cout<<" value of y";
cin>>y;

cout<<" Ans is "<< large (x, y);
```

the function `main`, the program terminates.

To put the ideas in this discussion to work, let us write a function that determines the larger of two numbers. Because the function compares two numbers, it follows that this function has two parameters and that both parameters are numbers. Let us assume that the data type of these numbers is floating-point (decimal)—say, **double**. Because the larger number is of type **double**, the function's data type is also **double**. Let us name this function `larger`. The only thing you need to complete this function is the body of the function. Thus, following the syntax of a function, you can write this function as follows:

```
double larger(double x, double y)
{
    double max;

    if (x >= y)
        max = x;
    else
        max = y;

    return max;
}
```

Note that the function `larger` requires that you use an additional variable `max` (called a **local declaration**, in which `max` is a variable local to the function `larger`). Figure 6-1 describes various parts of the function `larger`.

Suppose that num, num1, and num2 are **double** variables. Also suppose that num1 = 45.75 and num2 = 35.50. Figure 6-2 shows various calls to the function `larger`.

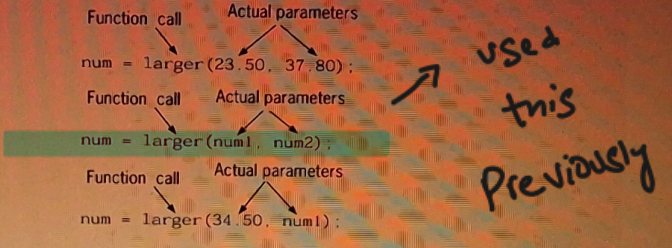


FIGURE 6-2 Function calls

You can also write the definition of the function `larger` as follows:

```
double larger(double x, double y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```

Because the execution of a `return` statement in a function terminates the function, the preceding function `larger` can also be written (without the word `else`) as:

```
double larger(double x, double y)
{
    if (x >= y)
        return x;

    return y;
}
```

Note that these forms of the function `larger` do not require you to declare any local variable

E.g Write a function that compares three numbers and outputs the largest.

```
int comparetree(int a, int b, int c)
{
    if (a>b && a>c)
        return a;
    else if (b>a && b>c)
        return b;
    return c;
}

int x, y, z;
cout<<"Enter";
cin>>x>>y>>z;
cout<<"Ans is"<<comparetree(x, y, z);
```

Using function within a function

```
int Large (int a, int b)
{
    if (a > b)
      return a
    else
      return b
}

int three (
```

```
int x, y;
cout<< "value of x";
cin>>x;
cout<<" value of y";
cin>y;

cout<<"Ans is"<< large (x, y);
```

(function prototype) → heading of function without its body.

Syntax

function type function name (parameter list);

↑

ends with a Semi-colon.

e.g

```
int three (int a, int b, int c) → already defined the function.

int main() {

    int x, y, z
    cin>> x >> y >> z;
    cout<< "Ans is" << three (x, y, z);

}

int three ( int a , int b, int c)
{
        if (a>b && a>c)
            return a;
        else if (b>a && b>c)
            return b;
        else if (c>a && c>b)
            return c;
}
```

Value- returning functions

```
Int Secret (int x)
{
    if (x > 5)
      return x * 2;

    return x;

}
```

```
return x, y;
```

here only value of y is returned.

e.g
```
int freturn();
int main()
{
  cout<<"The Ans to be returned is "<<freturn();
}
int freturn()
{
    int num= 2;
    int x=3;
    return num, x;
}
```

Output Answer

```
The Ans to be returned is 3.
```

no parameters are needed when we aren't taking any inputs (in main). Just performing an action. / e.g outputting

---

note:: your cpp program always starts from main() function

Void functions → these functions have no return type

Syntax

```
void functionname ( formal parameter list)
{
    Statements
}
```

Void functions are called in an standalone statement itself and not in any expression / outputs

Syntax of formal parameter list for void functions::

(datatype &Variable, datatype &Variable)....

Function call Syntax::

functionname(actual parameter list);

There are two types of parameters

↙ Value parameter

↘ Reference parameters

Fotmal parameter recieves copy of data frim the corresponding actual parameter

Formal parameter recieves the location / Address of the corresponding actual parameter.

↓

&Variable becomes pass by reference

```
Void expfun (int one, int &two)
{
//
}
```

one → Value para

two → Reference para

note:: As soon as data is copied from Actual para → formal para there is no link b/w the two. now formal para has its own copy of data.

The value of Actual parameters in main() remain unaffected of whats is happening to formal parameters in different functions.

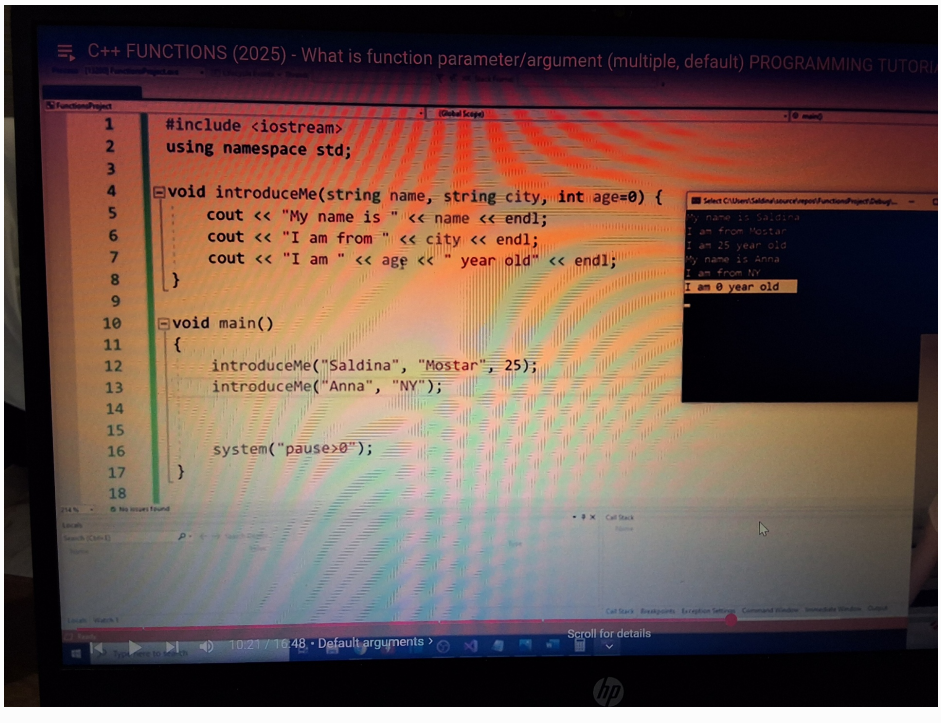
Reference parameters

Because we are passing the address of actual parameters to formal parameters

Any change to formal parameter also changes the value of actual parameter.

→ useful when value of actual parameter needs to be changed in int main.

default parameters



note: It is a must to give a default parameter at the end of the complete parameter list.

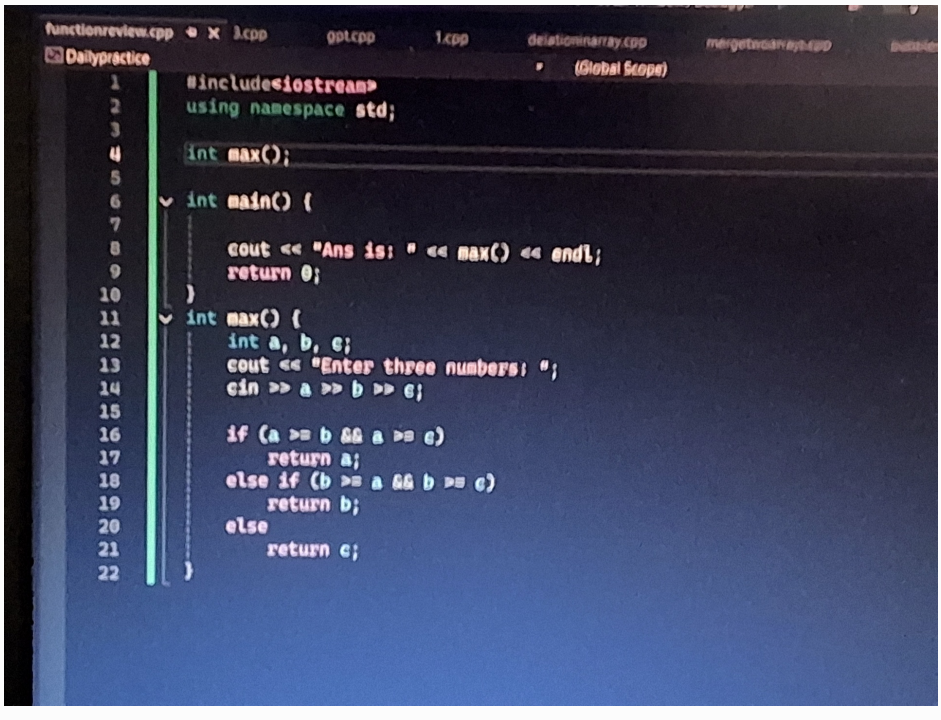
meaning basically they are assigned from Right → left

function overloading:: is when multiple functions have the same name but different parameters / different no of parameters / different types of parameters.

do Q27 from ch:: 6

note:: when taking inputs directly from our designed function we donot need actual parameters. Actual parameters are only needed when we are taking inputs from int main. /

Actual / formal parameters are only used when we are passing values into a function from int main. else if we are taking inputs directly from our function then no need.



Static variable:. → Stores the value of previous function call in itself.

```
int add (int x)
{
    static int sum=0;
    sum += x;
    return sum;
}

int main ()
{
    cout<< add(5)  → 5
    cout<< add(10) → 15
    cout<< add(15) → 30

}
```

Default parameters:.

When we have to pass values to a function explicity.

e.g

```
int sum (int x, int f = 1);
```

Note:. Default parameters are always passed in function prototype Before the function calling

```
int sum ( int x=0, int f=1)
int main ()
{
    cout<< sum ( 5, 6);

}
```

```
int Sum ( int x, int f)
{
    int Sum = x+f;
    return Sum;
}
```

output

Sum = 11

```
int sum ( int x=0, int f=1)
int main ()
{
    cout<< Sum ( 5 );
}
int Sum ( int x, int f)
{
    int Sum = x+f;
    return Sum;
}
```

output = 6

Default parameters are only used when there is no value in arguments (A.P

Always write default parameters from Right to left

int Sum (int x , int y , int z=0) ✓

int Sum (int x=0, int y =0, int z) ✗

int Sum (int x , int y=0 , int z) ✓

---

note: get line reads the entire input from the user including spaces.

getline (cin, Variable where we will store input)

File handling

#include <fstream>

contains definition for ifstream (Inputting data from a file) and ofstream (writing to a file)

opening a file e.g file name is furqan.txt

```
ofstream name("furqan.txt");
```

↳ used to write data in furqan.txt

for writing data we use <<

```
String st= "hello";
name<<st
```

→ stores hello in furqan.txt

To read data / Input data we use ifstream along with >>.

ofstream
↓
output use <<

ifstream
↓
Input use >>

Reading from furqan.txt

```
ifstream input("furqan.txt");
String name;
input >> name;
```

| getline(input, name)

↳ Reads the complete line stored in furqan.txt.

input.close() ← closes the file and we cannot read.

opening a file

filestream variable.open("file name.txt");

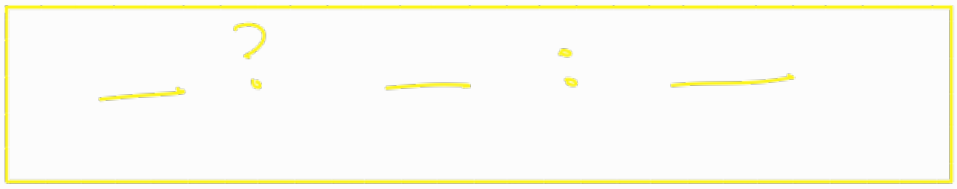
to check if file was opened successfully use

```
if (filestream variable.is_open())
```

closing a file

filestream variable.close();

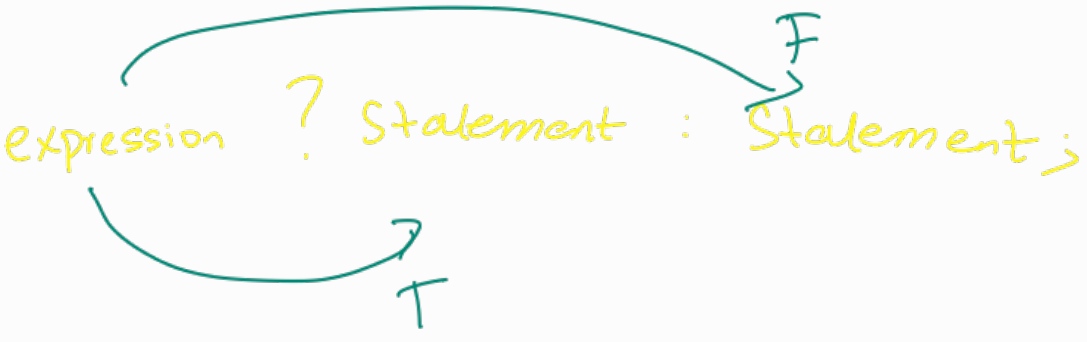
? tenary operator :- → consist of three parts



Syntax

```
x > y ? x : y ;
```

if x is greater than y then print x else print y.



To read data from a file. we have 3 cases:.

1) using getline (filestream var, var to Store);

→ this stores the complete line including spaces.

for (; getline (filestream var, var to Store);)

↳ reads the entire line till the end.

note:. var to Store must be of type string

→ to read one character at a time use

filestream var. get (variable to Store char)

variable must be of type char.

again use looping.

→ using cin >> directly this is good for reading formated code.

note cin = filestream variable

At 3.00
B- 2.67
C+ 06.4